PRODUCT CATALOG API

Project 1 — Beginner-Friendly Learning Report

Topics covered: Node.js, Express, TypeScript, REST APIs, HTTP, Zod, SKU uniqueness, service layers, middleware, error handling, filtering, searching, sorting, pagination.

A detailed reference to help you understand what you built, why each part exists, and what to learn next.

Project status snapshot

Area	Status based on our conversation
Server and health endpoint	Running; /health confirmed working.
Product operations	Create, list, get by ID, and delete routes implemented.
Validation	Zod schema validates create-product input.
Business rule	Duplicate SKU check added to in-memory service.
Error handling	Centralized Express error handler added and tested.
List query service	Step 5.2 service function added; route wiring and Postman verification are not yet confirmed.
Persistence	In-memory Map only; PostgreSQL is still a future step.

This report follows the work confirmed in our conversation. If your local code has changed further, use the report as a learning guide and verify implementation details in your repository.

1. What Are We Building?

A Product Catalog API is a backend service that lets another program create and manage product records. A frontend, mobile app, admin dashboard, or another service sends HTTP requests. The backend validates data, applies rules, performs the requested operation, and returns an HTTP response.

> An API is not the same as a website. The API exposes operations and data; a frontend may call those operations to display a user interface.

Example product record

```
{
  "id": "generated-uuid",
  "sku": "LAPTOP-001",
  "name": "Dell Latitude 5440",
  "price": 85000,
  "stock": 10,
  "category": "Electronics",
  "createdAt": "2026-10-25T04:00:00.000Z",
  "updatedAt": "2026-10-25T04:00:00.000Z"
}
```

The ID and timestamps are examples. Your application generates them; the actual values will differ.

2. Architecture: The Layers

A layer is a part of an application responsible for a particular kind of work. Separating responsibilities makes code easier to understand, test, and change.

Layer / file	Responsibility	Simple explanation
Client — Postman	Sends HTTP requests and displays responses.	The caller asking the API to do something.
src/server.ts	Starts listening on a port.	Turns the app on so clients can reach it.
src/app.ts	Creates Express app and registers middleware/routes.	Connects requests to handlers.
product.routes.ts	Maps methods/URLs to operations and HTTP responses.	Understands what operation was requested.
product.validation.ts	Defines Zod input rules.	Checks whether incoming data is acceptable.
product.service.ts	Product operations and business rules.	Decides what should happen.
product.types.ts	Defines Product TypeScript interface.	Describes expected product shape to developers/compiler.
error-handler.ts	Formats known and unexpected errors.	Central place for error responses.
Map	Stores records in process memory.	Temporary storage, lost on restart.
Future PostgreSQL	Persistent storage and constraints.	Keeps data across restarts and instances.

Simplified request lifecycle

```
Postman
   |
   v
Express app / middleware
   |
   v
Product route
   |
   +--> Zod validates the create request
   |
   v
Product service applies business rules
   |
   v
In-memory Map (temporary storage)
   |
   v
HTTP response (status code + JSON)
```

Errors can be forwarded to centralized error middleware. Not every request uses every layer; for example, a GET request does not normally validate a create-product body.

3. Core Backend Concepts

Backend

The backend is the server-side part of an application. It receives requests, runs application logic, reads or writes data, and sends responses. Users often interact with it indirectly through a frontend.

Node.js

Node.js is a JavaScript runtime: it lets JavaScript run outside a browser, including on a server. Node.js runs your Express application.

Express

Express is a web framework for Node.js. It provides routing and middleware tools so you do not have to build the entire HTTP request workflow yourself.

TypeScript

TypeScript adds static type checking to JavaScript. It helps catch mistakes while developing. Interfaces and types are removed from the compiled JavaScript; they do not validate incoming JSON at runtime.

REST API

REST is an architectural style commonly used to design network APIs around resources. Here, a product is a resource and /api/products is the collection endpoint. HTTP methods communicate the intended operation.

Method + endpoint	Purpose	Typical success status
GET /api/products	List products	200 OK
GET /api/products/:id	Get one product	200 OK
POST /api/products	Create product	201 Created
DELETE /api/products/:id	Delete product	204 No Content

HTTP request and response

A request includes a method, URL, headers, and sometimes a body. A response includes a status code, headers, and optionally a body. In Postman, choose a method, enter the URL, select Body → raw → JSON when sending JSON, then click Send.

JSON

JavaScript Object Notation is a text format for structured data. JSON keys and string values use double quotes. The value 85000 is a number; "85000" is a string, which may fail a numeric Zod rule.

Middleware

Middleware runs during the request-response cycle. It can inspect or change a request, send a response, or pass control onward. express.json() parses JSON bodies so route handlers can read req.body.

Route and route handler

A route connects an HTTP method and URL pattern to a function. For example, router.get('/:id', ...) handles GET requests for a product ID. The handler reads request data, calls the relevant logic, and returns a response or forwards an error.

4. HTTP Status Codes and API Errors

Code	Meaning	Use in this project
200	OK	A GET operation succeeded.
201	Created	A product was created.
204	No Content	Deletion succeeded; response body is empty.
400	Bad Request	Submitted data fails validation.
404	Not Found	Requested product ID does not exist.
409	Conflict	SKU conflicts with an existing product.
500	Internal Server Error	Unexpected server-side error.

Centralized error-handling middleware

Express error middleware has four parameters: (error, req, res, next). It gives the application one place to convert known errors into predictable responses. Your handler maps ZodError to 400, DuplicateSkuError to 409, and unexpected errors to 500.

Why keep errors consistent?

A consistent response shape helps frontend developers and other services handle errors without special-case parsing for every endpoint. A safe public message should be returned to clients while diagnostic details are logged on the server. Never expose secrets or sensitive configuration in error responses.

next(error)

Calling next(error) forwards an error through Express's error-handling pipeline. Express skips ordinary handlers and looks for error-handling middleware. Async error behavior depends on Express version and how asynchronous work is written.

Business rule vs. validation rule

Type	Example	Where handled
Input validation	Price is nonnegative; stock is an integer.	Zod schema
Business rule	Two products cannot share the same SKU.	Service; database constraint later
HTTP behavior	Duplicate SKU returns 409.	Route/error handler

5. Product Model, SKU, and Zod

Product interface

The Product interface describes the expected fields: id, sku, name, price, stock, category, createdAt, and updatedAt. It helps TypeScript check code, but it does not create a database table or validate external HTTP input.

```
export interface Product {
  id: string;
  sku: string;
  name: string;
  price: number;
  stock: number;
  category: string;
  createdAt: string;
  updatedAt: string;
}
```

SKU (Stock Keeping Unit)

A SKU is a business identifier used to distinguish a sellable product or variant for inventory operations. For example, LAPTOP-001 could identify a laptop item. Different variants, such as a T-shirt in different sizes, often need separate SKUs.

Field	Purpose	Example
id	Technical identifier used in API URLs and relationships.	Generated UUID
sku	Business identifier used in inventory, orders, purchasing, and reports.	LAPTOP-001

Why normalize SKU?

The service trims surrounding whitespace and converts the SKU to uppercase before checking for duplicates. This treats laptop-001 and LAPTOP-001 as equivalent. If a duplicate exists, the service throws DuplicateSkuError.

Production note: an application-level check alone is not enough under concurrent requests. PostgreSQL should enforce SKU uniqueness with a UNIQUE constraint as the final protection.

Zod

Zod is a TypeScript-friendly validation library. A schema defines rules and validates real input at runtime. Data from Postman, browsers, or other services is untrusted even when the application code uses TypeScript.

```
export const createProductSchema = z.object({
  sku: z.string().trim().min(1).max(50),
  name: z.string().trim().min(2).max(150),
  price: z.number().finite().nonnegative(),
  stock: z.number().int().nonnegative(),
  category: z.string().trim().min(2).max(80)
});
```

6. Understanding the Zod Rules

Rule	Meaning
z.object({...})	Input must be an object with fields governed by the schema.
z.string()	Value must be a string.
.trim()	Removes leading/trailing whitespace during parsing.
.min(n) / .max(n)	Sets string length limits in this schema.
z.number().finite()	Requires a finite number; rejects NaN and infinities.
.nonnegative()	Number must be 0 or greater.
.int()	Number must be an integer, such as 0, 1, or 10.
schema.parse(value)	Validates and returns parsed data, or throws ZodError.
z.infer	Derives a TypeScript type from the schema.

Type checking vs runtime validation

TypeScript checks code during development/build. Zod checks actual values while the app runs. If a client sends stock: 2.5, TypeScript cannot stop that JSON from arriving; the Zod schema rejects it because stock must be an integer.

Service layer

A service contains application/business logic independently of HTTP details. Your service creates products, lists products, finds by ID, deletes products, and checks SKU uniqueness. This separation makes logic easier to reuse and test.

In-memory Map

Map is a JavaScript key-value collection. Your product ID is the key and the product object is the value. It is useful for learning but not persistent: a process restart clears it, and separate server instances do not share it.

```
const products = new Map<string, Product>();
products.set(product.id, product); // store
products.get(id);                  // retrieve
products.delete(id);               // remove
```

UUID

A UUID is an identifier format designed to make accidental collisions extremely unlikely. randomUUID() generates a UUID for a product. A UUID is not an authorization mechanism; a real application must still check permissions.

7. Filtering, Searching, Sorting, and Pagination — Step 5.2

The listProducts() service function was added as the next step. It operates on in-memory products and is designed to return a page of results plus metadata. The route wiring and Postman verification have not yet been confirmed in our conversation.

Query parameters

Query parameters are optional values after ? in a URL. They let clients control the returned list without changing the endpoint.

```
GET /api/products?category=Electronics&search=laptop
GET /api/products?sortBy=price&order=asc
GET /api/products?page=1&limit=10
```

Filtering

Filtering narrows records by a condition. category=Electronics returns products in that category. Filtering should happen before pagination so the page contains matching records.

Searching

Searching matches a term against selected fields. The current design searches product name or SKU case-insensitively. includes() is suitable for a small exercise; production search needs careful query design and possibly indexes or dedicated search features.

Sorting

Sorting orders results by a selected field such as price or createdAt. asc means ascending; desc means descending. Allowlist sort fields rather than accepting arbitrary field names from clients.

Pagination

Pagination returns a manageable subset of records. page is the page number and limit is the maximum records per page. The offset is (page - 1) × limit. Page 2 with limit 10 skips the first 10 results and returns the next 10.

Field	Definition
page	Requested page number.
limit	Maximum records returned in this page.
total	Number of records matching filters/search before pagination.
totalPages	Ceiling of total divided by limit.

Correct processing order

```
1. Start with candidate products
2. Apply category filter
3. Apply search term
4. Sort matching results
5. Count matching records (total)
6. Slice the requested page
7. Return data + pagination metadata
```

8. File Map: Where Each Responsibility Lives

File	Responsibility
src/server.ts	Starts the HTTP server and chooses the port.
src/app.ts	Creates Express app, registers express.json(), health route, product routes, and error middleware.
src/modules/products/product.types.ts	Product TypeScript interface.
src/modules/products/product.validation.ts	Zod schema and inferred CreateProductInput type.
src/modules/products/product.service.ts	In-memory Map and product operations/business logic, including SKU uniqueness and listProducts().
src/modules/products/product.routes.ts	HTTP routes and response handling; listProducts() still needs to be connected to GET /api/products and tested.
src/middleware/error-handler.ts	Centralized error response handling.

9. Progress: Done vs. Remaining

Feature	Status	Next action
Server + /health	Confirmed working	Keep as health check.
Create/list/get/delete	Implemented	Re-test after route changes.
Zod validation	Implemented	Later validate query parameters too.
Duplicate SKU check	In-memory check	Add PostgreSQL UNIQUE constraint later.
Central error handler	Added; user reported done	Ensure all routes use it consistently.
List query service	Added in Step 5.2	Wire to GET /api/products and test.
PostgreSQL persistence	Not done yet	Next major milestone.
Automated tests	Not confirmed	Add unit and API tests.
OpenAPI/Swagger docs	Not done yet	Document routes and schemas.
Deployment/README	Not confirmed	Finish after core features.

10. Beginner Practice Checklist

- I can explain the difference between Node.js, Express, and TypeScript.
- I can explain method, URL, request body, response, and status code.
- I understand why GET retrieves data, POST creates data, and DELETE removes data.
- I can explain why TypeScript interfaces do not validate JSON at runtime.
- I can explain what Zod parse() does and why it can throw ZodError.
- I can explain why SKU and id are different.
- I understand why a PostgreSQL UNIQUE constraint is needed in addition to the service check.
- I can explain Express error middleware and next(error).
- I can explain filtering vs searching vs sorting vs pagination.
- I can explain why filtering occurs before pagination and what totalPages means.
- I understand why Map is temporary and why the project needs PostgreSQL.

11. Quick Glossary

Term	Simple definition
API	A defined way for software systems to communicate.
Endpoint	A particular HTTP method + URL that performs an operation.
Payload/request body	Data sent with a request, often JSON.
Schema	Rules describing the expected shape of data.
Validation	Checking whether data satisfies rules.
Business logic	Rules defining how the application should behave.
Persistence	Keeping data after the current process stops.
Constraint	A database rule that protects data integrity.
Index	A database structure that can speed up some lookups, with storage/write costs.
Transaction	A group of database operations treated as one unit, with commit or rollback.
Idempotency	Repeating an operation has the same intended effect as doing it once; useful for safe retries.
Concurrency	Multiple operations progressing at overlapping times.
Environment variable	Configuration outside source code, such as PORT or DATABASE_URL.
Unit test	A test focused on a small function/component in isolation.
Integration test	A test checking multiple components working together.
OpenAPI	A standard format describing HTTP APIs; Swagger UI can display interactive docs.

12. Recommended Next Steps

- **Finish Step 5:** connect listProducts() to GET /api/products; validate page, limit, sortBy, and order; test query combinations.
- **Learn PostgreSQL basics:** tables, primary keys, unique constraints, data types, SELECT/INSERT/UPDATE/DELETE, and migrations.
- **Replace Map storage:** keep the route/service separation but make the service read and write PostgreSQL.
- **Test essentials:** valid creation, invalid payload, duplicate SKU, missing product, filtering, and pagination.
- **Timebox:** write down advanced enhancements for later instead of delaying the next project.

13. Self-Test Questions

1. What does Express provide?
Routing and middleware abstractions for building web APIs on Node.js.

2. Why Zod if we already use TypeScript?
TypeScript checks code during development; Zod validates real runtime input.

3. Why enforce unique SKU in PostgreSQL?
Concurrent requests can race past an application check; a database constraint protects data integrity.

4. Difference between 400 and 409?
400 means invalid input; 409 means a conflict with existing state, such as a duplicate SKU.

5. Why filter before pagination?
So the requested page contains the correct matching records and total counts.

6. Why is Map not production persistence?
It is lost on process restart and is not shared among multiple server instances.

7. What does next(error) do?
Forwards an error through Express's error-handling pipeline.

8. What belongs in the service layer?
Business operations and rules, separated from HTTP request/response details.

Final takeaway

The key lesson is separation of responsibilities: routes handle HTTP, Zod validates external input, services enforce business rules, TypeScript types describe data for developers, middleware handles shared request/error behavior, and a database will provide durable storage and integrity constraints. Understanding those boundaries is more valuable than memorizing code.